

Section 6

Task 3

COMBINATIONAL DESIGN PRACTICES

19-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/FGfR>

GIT LINK: https://github.com/Nooorrr66/MEDS_2026/tree/main/module_4/6_Combinational_Design_Practices

CODE:

Test Bench

```
module testbench;

logic a;
logic b;
logic cin;

logic sum1;
logic carry1;

logic sum2;
logic carry2;

logic sum;
logic cout;

// Half Adder using assign

half_adder_assign HA1(

.a(a),
.b(b),
.sum(sum1),
.carry(carry1)

);

// Half Adder using always_comb

half_adder_always HA2(

.a(a),
.b(b),
.sum(sum2),
.carry(carry2)

);
```

Design:

```
// Half Adder using assign
module half_adder_assign(
    input logic a,
    input logic b,
    output logic sum,
    output logic carry

);
    assign sum = a ^ b;
    assign carry = a & b;

endmodule

// Half Adder using always_comb
module half_adder_always(
    input logic a,
    input logic b,
    output logic sum,
    output logic carry

);
    always_comb begin

        if(a == b)
            sum = 0;
        else
            sum = 1;

        if(a == 1 && b == 1)
            carry = 1;
        else
            carry = 0;

    end

endmodule
```

```
// Full Adder
```

```
full_adder FA(
```

```
.a(a),  
.b(b),  
.cin(cin),  
.sum(sum),  
.cout(cout)
```

```
);
```

```
initial begin
```

```
$dumpfile("dump.vcd");  
$dumpvars(0, testbench);  
$display("Half Adder");
```

```
a=0; b=0; #10;  
$display("%b %b -> %b %b | %b  
%b",a,b,sum1,carry1,sum2,carry2);
```

```
a=0; b=1; #10;  
$display("%b %b -> %b %b | %b  
%b",a,b,sum1,carry1,sum2,carry2);
```

```
a=1; b=0; #10;  
$display("%b %b -> %b %b | %b  
%b",a,b,sum1,carry1,sum2,carry2);
```

```
a=1; b=1; #10;  
$display("%b %b -> %b %b | %b  
%b",a,b,sum1,carry1,sum2,carry2);
```

```
$display("");  
$display("Full Adder");
```

```
a=0; b=0; cin=0; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=0; b=0; cin=1; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=0; b=1; cin=0; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=0; b=1; cin=1; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=1; b=0; cin=0; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=1; b=0; cin=1; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=1; b=1; cin=0; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
a=1; b=1; cin=1; #10;  
$display("%b %b %b -> %b %b",a,b,cin,sum,cout);
```

```
$finish;
```

```
end
```

```
endmodule
```

```
// Full Adder
```

```
module full_adder(
```

```
input logic a,  
input logic b,  
input logic cin,
```

```
output logic sum,  
output logic cout
```

```
);
```

```
logic s;  
logic c1;  
logic c2;
```

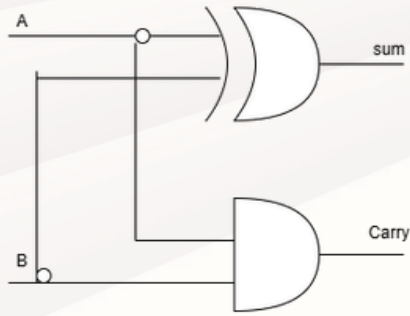
```
half_adder_assign h1(  
.a(a),  
.b(b),  
.sum(s),  
.carry(c1)  
);
```

```
half_adder_assign h2(  
.a(s),  
.b(cin),  
.sum(sum),  
.carry(c2)  
);
```

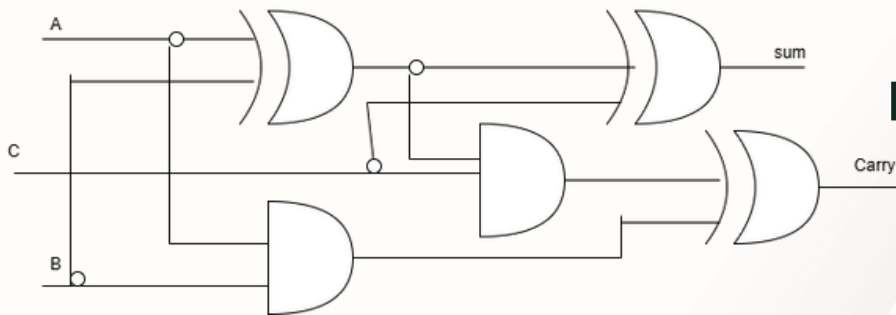
```
assign cout = c1 | c2;
```

```
endmodule
```

SCHEMATIC:



HALF ADDER



FULL ADDER

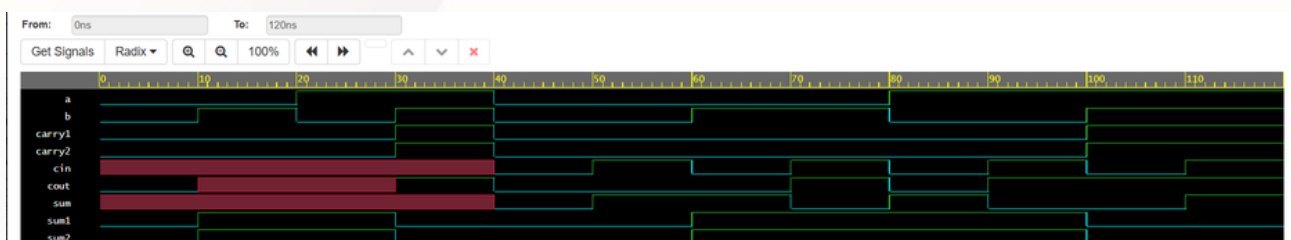
TERMINAL:

```
Log Share
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64; Jul 19 11:51 2026

Half Adder
0 0 -> 0 0 | 0 0
0 1 -> 1 0 | 1 0
1 0 -> 1 0 | 1 0
1 1 -> 0 1 | 0 1

Full Adder
0 0 0 -> 0 0
0 0 1 -> 1 0
0 1 0 -> 1 0
0 1 1 -> 0 1
1 0 0 -> 1 0
1 0 1 -> 0 1
1 1 0 -> 0 1
1 1 1 -> 1 1
$finish called from file "testbench.sv", line 100.
```

WAVEFORM:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

EXPLANATION:

Two half adders were implemented using different coding styles: one with assign statements and the other with an always_comb block. A full adder was then built by connecting two half adders and an OR gate. The design was simulated to verify the correct outputs.

The simulation tested all input combinations. Both half adder implementations produced identical outputs for every test case. The full adder also generated the correct sum and cout values for all eight input combinations.

The waveform shows how the input signals a, b, and cin change over time during the simulation.

As the inputs change, the outputs sum1 and sum2 always change together, showing that both half adder implementations behave identically. The signals carry1 and carry2 also match for every input combination.

The waveform also shows the outputs sum and cout of the full adder. These outputs change according to the values of a, b, and cin, confirming the correct operation of the structural full adder.

The red (X) values visible at the beginning of the waveform represent unknown values before the inputs are initialized. Once the inputs are assigned, all signals settle to their correct logic values.

Although the two half adders are written using different coding styles, they describe the same Boolean logic. The assign version directly specifies the Boolean equations, while the always_comb version uses procedural if-else statements to produce the same outputs. During synthesis, the synthesis tool analyzes the logic rather than the coding style. Since both modules implement the same truth table, they are synthesized into the same hardware consisting of one XOR gate for the sum output and one AND gate for the carry output.

Task 4

19-07-2026

PLAYGROUND LINK:

<https://www.edaplayground.com/x/Sx7E>

CODE:

Test Bench

```
`timescale 1ns/1ps

module testbench;
logic a;
logic b;
logic c;
logic d;
logic f1;
logic f2;
integer i;

// Direct Module
nor_direct D1(
.a(a),
.b(b),
.c(c),
.d(d),
.f(f1)
);

// NOR Module
nor_only D2(
.a(a),
.b(b),
.c(c),
.d(d),
.f(f2)
);

initial begin
$dumpfile("dump.vcd");
$dumpvars(0,testbench);
$display("a b c d | Direct| only ");

for(i=0;i<16;i=i+1)

begin
a = i[3];
b = i[2];
c = i[1];
d = i[0];

#10;
$display("%b %b %b %b | %b %b",
a,b,c,d,f1,f2);

if(f1==f2)
$display("PASS");

else
$display("FAIL");

end

$finish;
end
endmodule
```

Design:

```
// Code your design here
`timescale 1ns/1ps

// Direct Implementation
module nor_direct(
input logic a,
input logic b,
input logic c,
input logic d,
output logic f
);

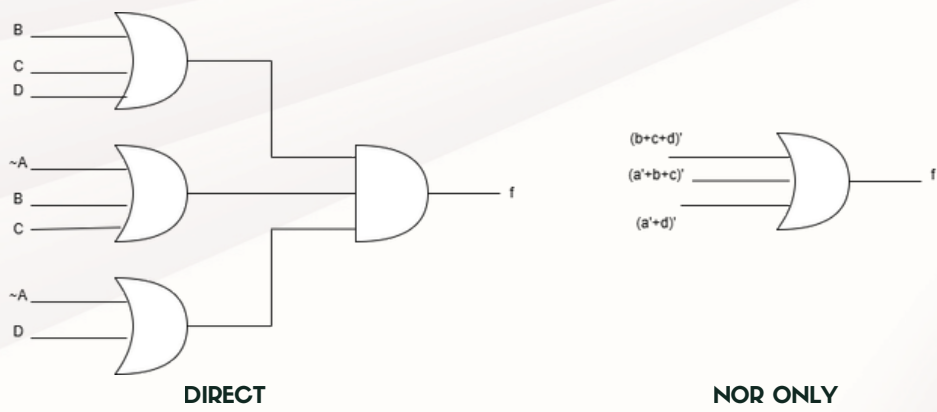
assign f = (b | c | d) &
(~a | b | c) &
(~a | d);
endmodule

// NOR Only Implementation
module nor_only(
input logic a,
input logic b,
input logic c,
input logic d,
output logic f
);

logic x;
logic y;
logic z;
assign x = ~(b | c | d);
assign y = ~(~a | b | c);
assign z = ~(~a | d);
assign f = ~(x | y | z);

endmodule
```

SCHEMATIC:



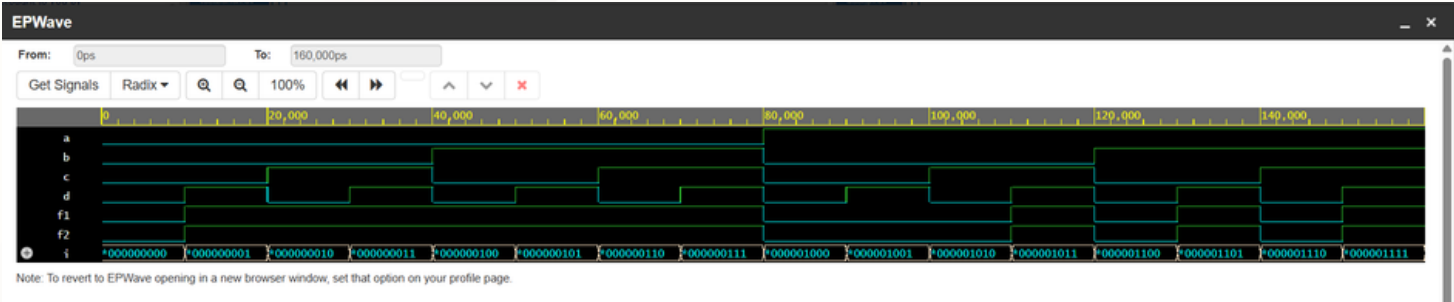
TERMINAL:

LogShare

Chronologic VCS Simulator Copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 20 01:54 2026

```
a b c d | Direct| only
0 0 0 0 | 0 0
PASS
0 0 0 1 | 1 1
PASS
0 0 1 0 | 1 1
PASS
0 0 1 1 | 1 1
PASS
0 1 0 0 | 1 1
PASS
0 1 0 1 | 1 1
PASS
0 1 1 0 | 1 1
PASS
0 1 1 1 | 1 1
PASS
1 0 0 0 | 0 0
PASS
1 0 0 1 | 0 0
PASS
1 0 1 0 | 0 0
PASS
1 0 1 1 | 1 1
PASS
1 1 0 0 | 0 0
PASS
1 1 0 1 | 1 1
PASS
1 1 1 0 | 0 0
PASS
1 1 1 1 | 1 1
PASS
$finish called from file "testbench.sv", line 77.
$finish at simulation time          160000
```

WAVEFORM:



EXPLANATION:

Task 4

$$f = (b+c+d)(a'+b+c)(a'+d)$$
$$x = (b+c+d); y = (a'+b+c); z = (a'+d)$$
$$f = x \cdot y \cdot z$$

By demorgan's law $\div A \cdot B \cdot C = (A' + B' + C')'$

$$f = (x' + y' + z')'$$
$$x' = (b+c+d)'; y' = (a'+b+c)'; z' = (a'+d)'$$
$$f = ((b+c+d)' + (a'+b+c)' + (a'+d)')$$

$\rightarrow$ NOR only expression.

Two implementations of the given Boolean expression were created. The first uses the original Boolean expression directly with an assign statement. The second uses only NOR operations obtained using De Morgan's Law. Both modules were tested to verify that they produce identical outputs.

The testbench checks all 16 combinations of a, b, c, and d. The outputs of both implementations match for every test case, and every comparison prints PASS, confirming that both circuits are equivalent.

The waveform shows all input combinations and the corresponding outputs. The signals f1 and f2 overlap throughout the simulation, proving that the direct implementation and the NOR-only implementation behave identically.

The loop variable i automatically generates these combinations by counting from 0000 to 1111.

Task 5

19-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/hgMz>

CODE:

Test Bench

```
`timescale 1ns/1ps

module testbench;
logic [3:0] data;
logic parity;
logic expected;
integer i;

// Instantiate the parity generator
parity_generator DUT(
.data(data),
.parity(parity)
);

initial begin
$dumpfile("dump.vcd");
$dumpvars(0,testbench);

$display("Data | Parity | Expected");

for(i=0; i<16; i=i+1)

begin
data = i;

#10;
expected = $countones(data) % 2;

$display("%b | %b | %b",
data, parity, expected);

if(parity == expected)
$display("PASS");
else
$display("FAIL");

end

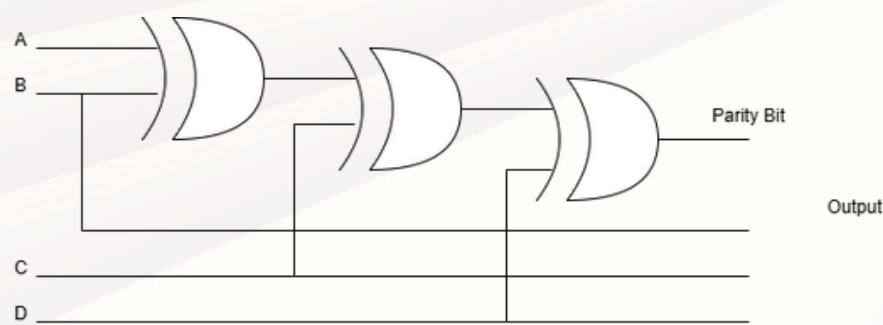
$finish;
end
endmodule
```

Design:

```
`timescale 1ns/1ps
module parity_generator(
input logic [3:0] data,
output logic parity
);

assign parity = ^data;
endmodule
```

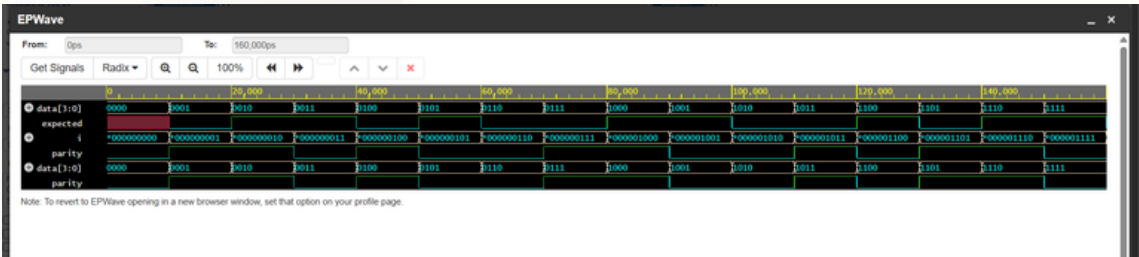

SCHEMATIC:



TERMINAL:

```
Log Share
Copyright Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 20 02:59 2026
Data | Parity | Expected
0000 | 0 | 0
PASS
0001 | 1 | 1
PASS
0010 | 1 | 1
PASS
0011 | 0 | 0
PASS
0100 | 1 | 1
PASS
0101 | 0 | 0
PASS
0110 | 0 | 0
PASS
0111 | 1 | 1
PASS
1000 | 1 | 1
PASS
1001 | 0 | 0
PASS
1010 | 0 | 0
PASS
1011 | 1 | 1
PASS
1100 | 0 | 0
PASS
1101 | 1 | 1
PASS
1110 | 1 | 1
PASS
1111 | 0 | 0
PASS
$finish called from file "testbench.sv", line 41.
```

WAVEFORM:



EXPLANATION:

A 4-bit parity generator was implemented using a single assign statement with the reduction XOR operator ($\wedge$). The output becomes 1 when the input contains an odd number of 1s and 0 otherwise.

The `$countones()` function is a built-in SystemVerilog function that counts the number of bits set to 1 in the input data. In this task, it is used to calculate the expected parity value for comparison with the module output. The expression `$countones(data) % 2` checks whether the number of 1s in the 4-bit input is odd or even. If the remainder is 1, the input contains an odd number of 1s, so the expected parity is 1. If the remainder is 0, the number of 1s is even, so the expected parity is 0. This provides a simple and reliable way to verify that the parity generator produces the correct output for all 16 input combinations. The testbench checks all 16 possible input combinations. For each input, the expected parity is calculated using `$countones(data) % 2` and compared with the module output. Every test passes, confirming the correctness of the design.

The waveform shows the 4-bit input data changing sequentially through all 16 possible binary values during the simulation. As the input changes, the parity output is updated automatically by the reduction XOR operator. The expected signal, calculated using `$countones(data) % 2`, follows the same pattern as the generated parity output. Throughout the simulation, both signals remain identical, confirming that the implementation is correct. The initial unknown (X) value on the expected signal occurs before it is assigned a value in the testbench and is a normal part of digital simulation. Once the simulation begins, both parity and expected produce matching outputs for every input combination, verifying the correct operation of the 4-bit parity generator.